

Brno University of Technology
Faculty of Information Technology

Image Processing Filter Effects

Project Report

ZPOe 25/26

Adar Otieno - xotiena00
Deniz Aksoy - xaksoyd00
Eyobed Awel Nuri - xnuriey00

May 6, 2026

Contents

1 Introduction **3**
1.1 Project Architecture 3

2 Implementation Details **3**
2.1 Eyobed's Filters 3
2.2 Adar's Filters 4
2.3 Denis's Filters 4

3 Graphical User Interface **4**

4 Conclusion **5**

1 Introduction

In the field of computer vision and image processing, filters are essential tools used to manipulate and analyze images. They operate by modifying the pixels of an image based on mathematical functions, allowing for enhancements, artistic effects, or noise reduction.

This project explores the implementation of custom image filters using C++ and the OpenCV library. Rather than relying entirely on pre-built, high-level OpenCV functions, the focus of this work is on the manual, pixel-by-pixel implementation of various effects to understand the underlying mathematics and algorithms. The application serves as an interactive GUI that allows users to seamlessly apply and switch between 12 unique filters developed by the team.

1.1 Project Architecture

The core application is built as a single interactive window utilizing OpenCV's HighGUI module. The architecture emphasizes modularity, ensuring that each contributor can develop their effects independently.

The application utilizes a `Filter` structure to register the effects:

```
struct Filter {
  string author;
  string name;
  string description;
  Mat (*apply)(const Mat &);
};
```

This structure enables the main application to dynamically load the filters, populate the trackbar controls, and render an informative metadata panel detailing the active effect and its creator.

2 Implementation Details

The project incorporates a total of 12 distinct filters, divided equally among the three contributors: Eyobed, Adar, and Denis.

2.1 Eyobed's Filters

- **Histogram Equalization:** Enhances image contrast by redistributing the brightness values across the full 0-255 range. To prevent unwanted color shifting, the algorithm operates in the YCrCb color space, applying the equalization solely to the luminance (Y) channel before converting back to BGR.
- **Lens Distortion:** Applies a manual barrel distortion effect, simulating the physical curvature of a camera lens. The pixels are mapped using a radial geometric transform.
- **Zoom Blur:** A radial blur effect that streaks pixels from the edges of the image towards the center. This is achieved using a manually implemented directional box blur kernel.

- **Posterize:** Reduces the continuous color palette into a limited number of discrete values. This manual color quantization gives the image a distinct comic-book or retro-art aesthetic.

2.2 Adar's Filters

Adar's contributions focus on manual pixel manipulation using native loops, avoiding optimized OpenCV functions for educational purposes.

- **Relief Emboss:** Creates a 3D gray relief texture by subtracting the bottom-right neighboring pixel from the top-left neighbor and adding an offset of 128 to center the result.
- **Dodge:** Implements the classic Color Dodge blend formula: `(pixel * 255) / max(1, 255 - pixel)`. This blows out the highlights and drastically increases contrast.
- **Swirl:** Warps the image into a vortex by converting Cartesian coordinates to Polar coordinates, applying a nonlinear rotational offset based on the radius, and converting back to sample the pixel.
- **Bloom:** Extracts bright pixels (intensity > 180) and applies a brute-force nested 7x7 loop to blur them. The resulting glow is then added directly back to the original image.

2.3 Denis's Filters

Similar to Adar, Denis's filters prioritize foundational algorithms through direct matrix iteration.

- **Threshold:** Iterates through every pixel and manually calculates the luminosity using the `0.299R + 0.587G + 0.114B` formula, applying a hard check to push values to pure black or white.
- **Diffuse:** Creates a frosted glass effect by replacing each pixel's value with a randomly selected neighbor from within a localized radius.
- **Crosshatch:** Uses modulo arithmetic on the X and Y coordinates to draw diagonal pen strokes. The density of the strokes increases based on the darkness of the underlying pixel.
- **Hue Rotate:** Utilizes OpenCV strictly to convert the color space to HSV, then manually loops through the matrix applying modulo arithmetic to shift the Hue channel by +60 degrees.

3 Graphical User Interface

The application provides an intuitive way to visualize the image processing results. It is composed of three primary windows:

1. **Controls:** Houses two OpenCV trackbars. The first trackbar cycles through all available images in the `data/` directory. The second trackbar cycles through the 12 registered filters.

2. **Original / Filtered Views:** Displays the before-and-after results. To ensure a consistent viewing experience, a `resizeForDisplay()` function scales both images to fit within an 800x600 bounding box, preserving the aspect ratio. This scaling is done purely for presentation; the filters continue to process the native, full-resolution matrix.

3. **Info Panel:** An informative window that dynamically displays the name of the currently active filter, its creator, and a detailed description of how the mathematical effect is achieved.

4 Conclusion

This project successfully demonstrates the application of both fundamental and advanced image processing techniques. By implementing many of the effects manually through pixel iteration, the team gained a deep understanding of the mathematical operations that power modern computer vision libraries. The resulting application is highly modular, easily extensible, and serves as an excellent testbed for future algorithmic experiments. The authors used tools such as LLM to assist with development of the solution in accordance with the way permitted by the instructor at the beginning of the semester.